**BTS SIO — Option SISR**

Compte-rendu de TP

# Lab Nmap

*Cartographie et audit réseau en environnement professionnel*

| Champ | Valeur |
|---|---|
| Étudiant | Alexandre Marquant |
| Classe | BTS SIO 2 — option SISR |
| Environnement | Proxmox VE (nœud pve-j) — réseau de lab isolé |
| Cibles | Debian (auditeur), Ubuntu Server, Windows 11, Metasploitable2 |
| Date de réalisation | 15 octobre 2026 |
| Outils | Nmap 7.95, curl, Hydra 9.5, tcpdump 4.99 |

# 1. Contexte, cadre légal et maquette

## 1.1 Rappel du cadre légal et déontologique

Un scan de ports n'est pas un acte anodin : il laisse des traces dans les journaux de la cible et peut déclencher un système de détection d'intrusion (IDS/IPS). En droit français, l'article 323-1 du Code pénal réprime le fait d'accéder ou de se maintenir frauduleusement dans un système de traitement automatisé de données (STAD).

Règle d'or appliquée dans tout ce TP : je ne scanne que mes propres machines, dans un réseau de lab isolé. Mon binôme travaillant sur le segment 192.168.56.0/24, j'ai réalisé l'ensemble des manipulations sur mon propre segment 192.168.41.0/24 afin d'éviter tout conflit d'adressage et tout scan d'une machine qui ne m'appartient pas.

## 1.2 Maquette réalisée

Les quatre machines virtuelles sont reliées sur un même pont (vmbr4), sur le réseau interne isolé 192.168.41.0/24. Chacune dispose d'une IP fixe. La Debian joue le rôle du poste auditeur (équivalent Kali) ; Nmap y a été installé.

| Rôle | Système | IP | Remarques |
|---|---|---|---|
| Poste auditeur | Debian 13 | 192.168.41.1 | Nmap, curl, Hydra, tcpdump — + 2e carte vers Internet |
| Serveur applicatif | Ubuntu Server 24.04 | 192.168.41.2 | SSH + serveur web Apache |
| Poste client | Windows 11 | 192.168.41.3 | Pare-feu Windows Defender actif |
| Cible vulnérable | Metasploitable2 | 192.168.41.4 | Services obsolètes — jamais hors du lab |

## 2. TP 1 — Prise en main (cible : localhost)

Objectif : prendre en main la syntaxe de Nmap sur la machine auditeur elle-même, comparer la vision réseau de Nmap avec la vision système (ss), et produire un export exploitable.

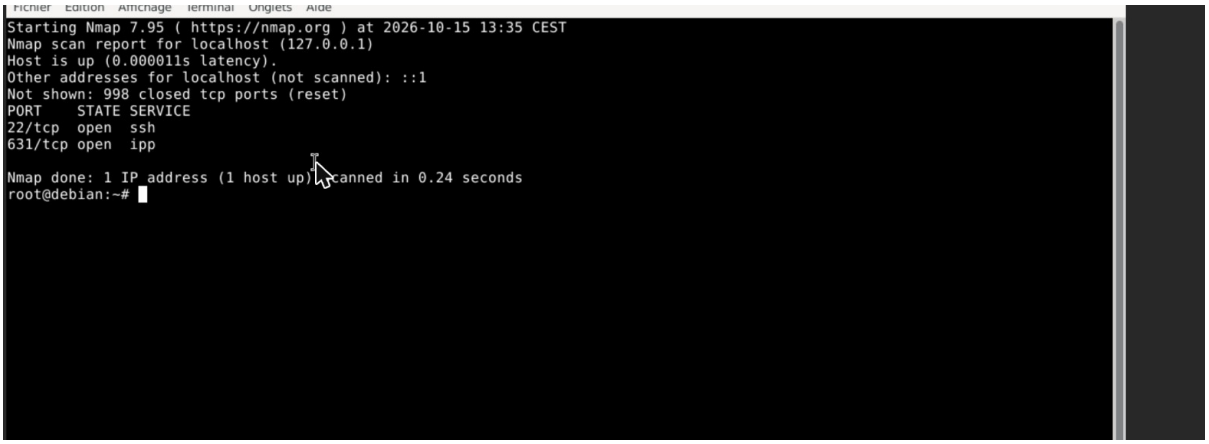
### Étape 0 — Version de Nmap

```
nmap --version
```

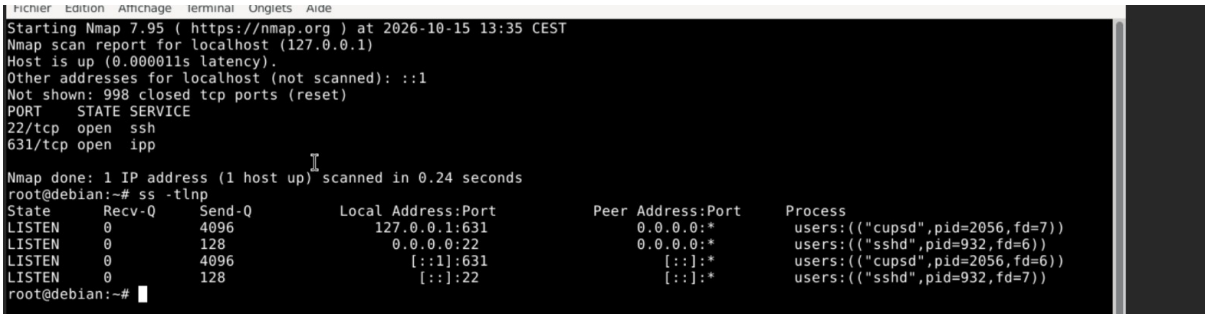
Nmap 7.95 est installé sur la Debian (installé via sudo apt install nmap).

### Étape 1 — Scan basique de la machine + comparaison avec ss

```
nmap localhost
ss -tlnp
```




*Scan de localhost : 22/ssh et 631/ipp (CUPS) ouverts.*




*ss -tlnp : sshd écoute sur 0.0.0.0:22, cupsd uniquement sur 127.0.0.1:631.*

Résultat : Nmap détecte deux ports ouverts, 22 (ssh) et 631 (ipp). La commande ss confirme que sshd écoute sur toutes les interfaces (0.0.0.0:22) alors que cupsd n'écoute qu'en local (127.0.0.1:631) : ce dernier n'est donc pas exposé au réseau.

### Étape 2 — Ports spécifiques

```
nmap -p 22,80,443 localhost
```

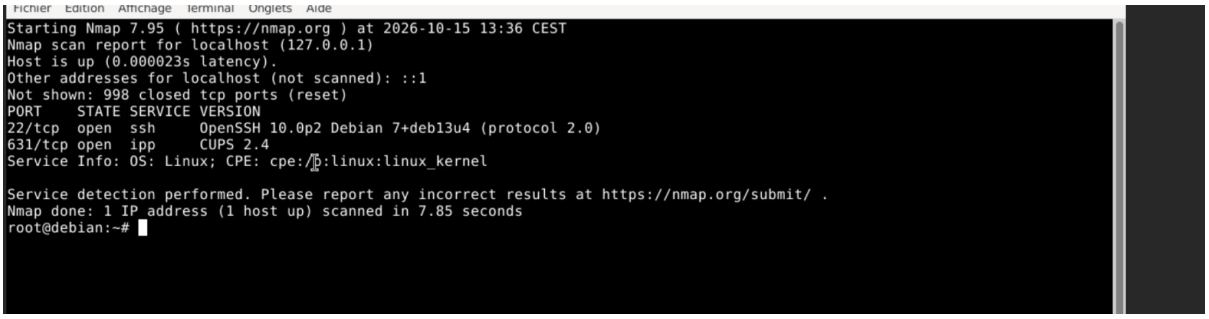



*22 open, 80 et 443 closed.*

Résultat : seul le port 22 est ouvert ; 80 et 443 sont fermés (closed) — la machine répond mais aucun service web n'écoute.

### Étape 3 — Détection de version

```
nmap -sV localhost
```

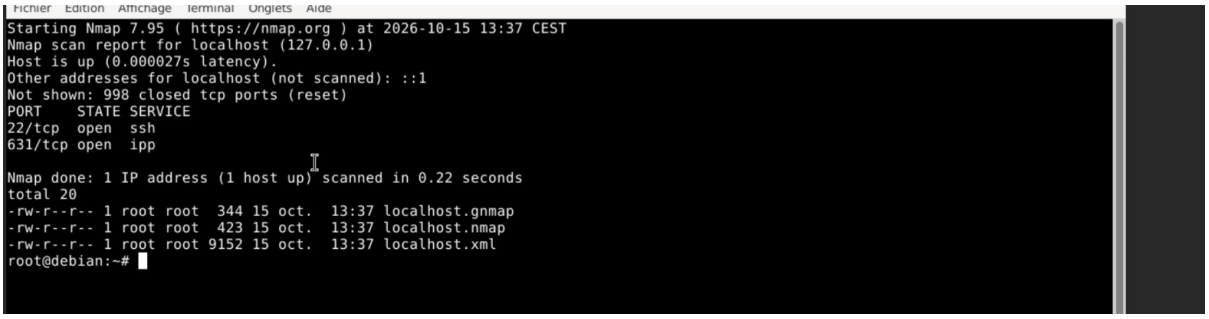



*Versions : OpenSSH 10.0p2 (Debian) et CUPS 2.4.*

Format de sortie : port / état / service / version. Nmap identifie OpenSSH 10.0p2 Debian 7+deb13u4 et CUPS 2.4.

### Étape 4 — Export multi-format

```
mkdir -p ~/nmap-lab/results
nmap -oA ~/nmap-lab/results/localhost localhost
```




*Trois fichiers générés : .nmap (lisible), .xml (outils tiers), .gnmap (grepable).*

L'option -oA génère simultanément les trois formats : .nmap (lecture humaine), .xml (réexploitable par d'autres outils) et .gnmap (filtrable en ligne de commande avec grep).

**Question de compréhension :** *pourquoi la sortie de nmap -sV localhost peut-elle différer légèrement de celle de ss -tlnp ?*

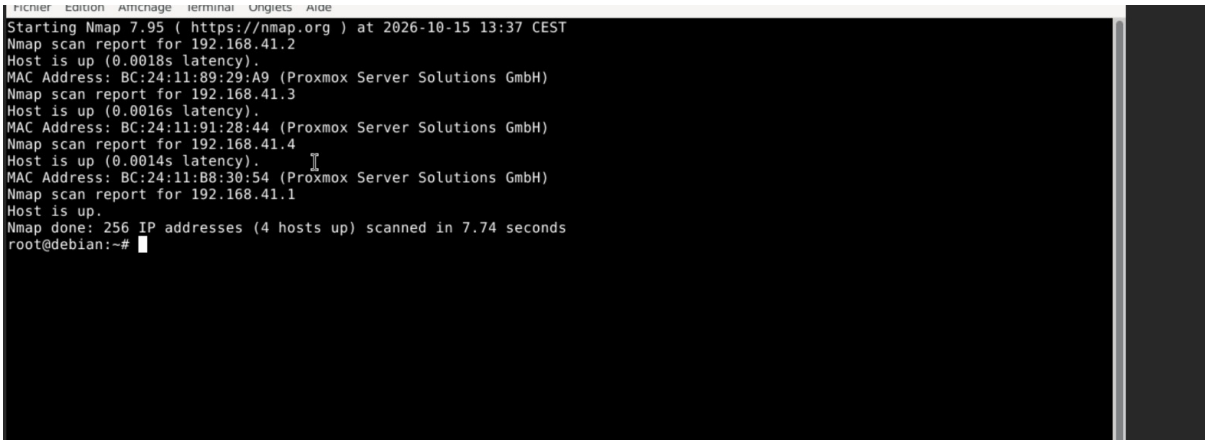
**Réponse.** Les deux outils ne regardent pas la même chose. ss lit directement la table des sockets du noyau : c'est la vérité sur ce qui écoute réellement en local. Nmap, lui, sonde depuis le réseau et déduit le service à partir des réponses et des bannières : il ne voit que ce qui est joignable sur l'interface scannée (ici 127.0.0.1) et peut nommer un service différemment ou donner une version approximative. ss est donc la référence côté système, Nmap la vision d'un attaquant côté réseau.

# 3. TP 2 — Cartographie du réseau et détection d'hôtes

Objectif : cartographier le segment 192.168.41.0/24, inventorier les hôtes actifs, puis mesurer l'impact de la résolution DNS sur la durée d'un scan.

### Étape 1 — Découverte d'hôtes (ping scan)

```
nmap -sn 192.168.41.0/24
```




*4 hôtes actifs découverts, avec leurs adresses MAC.*

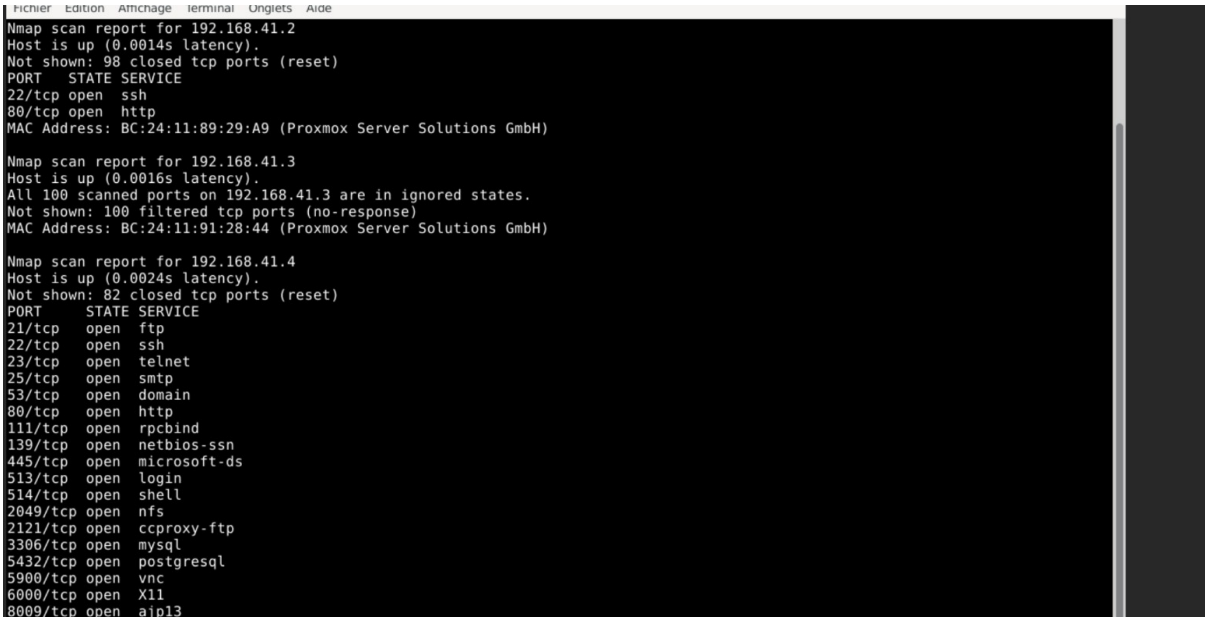
Le ping scan découvre les 4 machines du lab. Point important : le Windows (.3) est détecté alors qu'il bloque le ping ICMP — sur un réseau local, Nmap utilise l'ARP pour la découverte, qui ne peut pas être masqué par un pare-feu applicatif. Inventaire obtenu :

| IP | Adresse MAC | Hypothèse |
|---|---|---|
| 192.168.41.1 | (poste local) | Debian — poste auditeur |
| 192.168.41.2 | BC:24:11:89:29:A9 | Ubuntu Server |
| 192.168.41.3 | BC:24:11:91:28:44 | Windows 11 (ne répond pas au ping) |
| 192.168.41.4 | BC:24:11:B8:30:54 | Metasploitable2 |

Toutes les MAC commencent par BC:24:11, l'OUI de Proxmox/QEMU : cohérent avec un environnement 100 % virtualisé.

### Étape 2 — Scan rapide des hôtes trouvés

```
nmap -F -oA scan-initial 192.168.41.1-4
```




*Scan -F : .2 = ssh+http, .3 = 100 ports filtered (pare-feu), .4 = nombreux services.*

Le scan rapide (100 ports courants) dresse un premier profil : l'Ubuntu (.2) expose ssh et http, le Windows (.3) renvoie 100 ports filtered (pare-feu), et Metasploitable (.4) expose une vingtaine de services obsolètes.

### Étape 3 — Impact de la résolution DNS

```
time nmap -sn 192.168.41.0/24
time nmap -sn -n 192.168.41.0/24
```




*7,77 s avec DNS contre 2,15 s avec -n.*

Écart mesuré : 7,77 s avec résolution DNS contre 2,15 s avec l'option -n (soit ~3,6× plus rapide). Sur le lab il n'y a pas de serveur DNS pour les adresses privées : Nmap tente une résolution inverse sur chacune des 256 IP et attend l'expiration du délai à chaque fois. L'option -n supprime ces requêtes inutiles.

**Question de compréhension :** *pourquoi le ping scan (-sn) est-il l'étape recommandée avant tout scan de ports sur un réseau large ?*

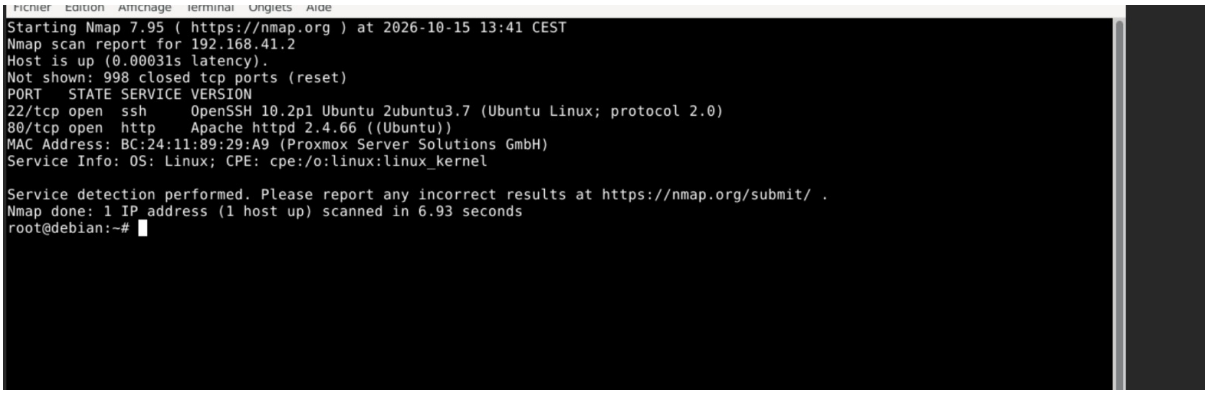
**Réponse.** Parce qu'il évite de gaspiller du temps à scanner les ports d'adresses mortes. Sur un /24, 256 adresses existent mais seules quelques-unes sont actives (ici 4). Le -sn identifie d'abord les hôtes vivants, de façon rapide et peu bruyante (ARP/ICMP), pour ne lancer ensuite les scans de ports — longs — que sur les machines réellement présentes. C'est la logique « du large vers le ciblé ».

# 4. TP 3 — Audit d'un serveur Ubuntu (192.168.41.2)

Objectif : auditer un serveur applicatif (SSH + web), recouper les informations, chercher des vulnérabilités, puis proposer une action de durcissement.

### Étape 1 — Détection de version

```
nmap -sV 192.168.41.2
```

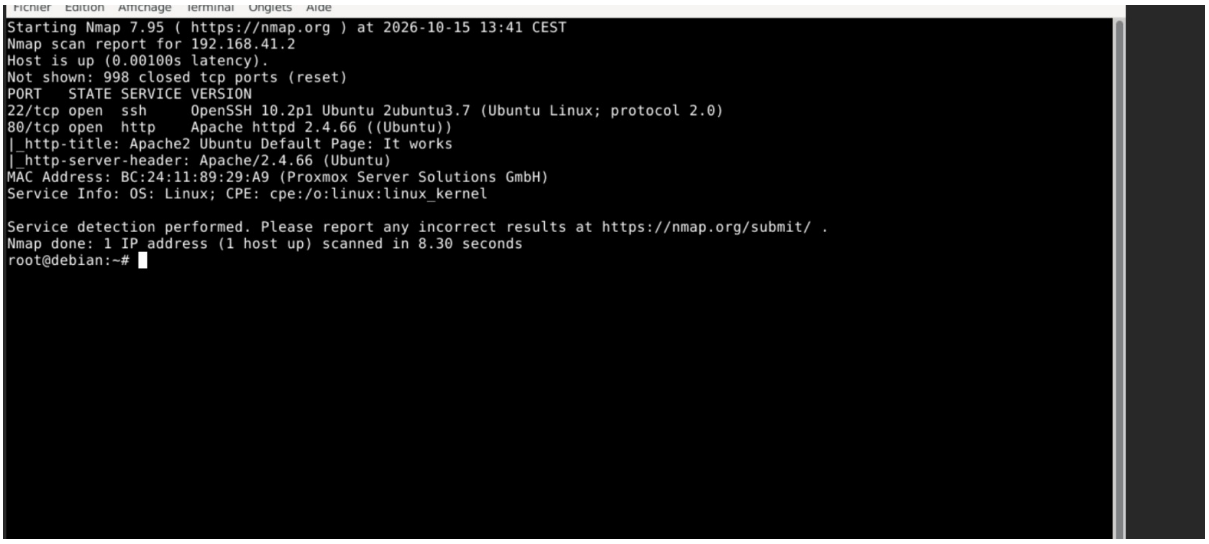



*OpenSSH 10.2p1 (Ubuntu) et Apache httpd 2.4.66.*

Deux services exposés : OpenSSH 10.2p1 Ubuntu (port 22) et Apache httpd 2.4.66 (port 80) — versions récentes.

### Étape 2 — Scripts par défaut

```
nmap -sC -sV 192.168.41.2
```




*Scripts NSE : titre et bannière HTTP révélés.*

Les scripts par défaut (-sC) révèlent des informations supplémentaires : http-title « Apache2 Ubuntu Default Page: It works » et http-server-header « Apache/2.4.66 (Ubuntu) ».

### Étape 3 — Vérification manuelle (recoupement)

```
curl -I http://192.168.41.2
```

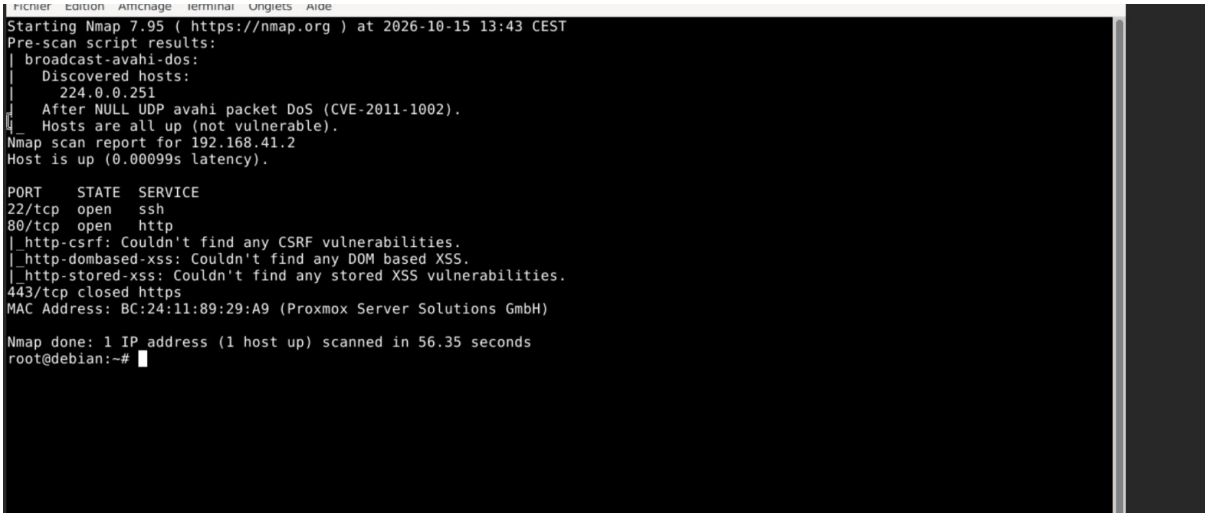



*En-tête HTTP : Server: Apache/2.4.66 (Ubuntu) — cohérent avec Nmap.*

Le recoupement avec curl confirme l'en-tête « Server: Apache/2.4.66 (Ubuntu) » annoncé par Nmap : bonne pratique d'auditeur — toujours recouper une détection automatique par une vérification manuelle.

### Étape 4 — Recherche de vulnérabilités

```
nmap --script vuln -p 22,80,443 192.168.41.2
```

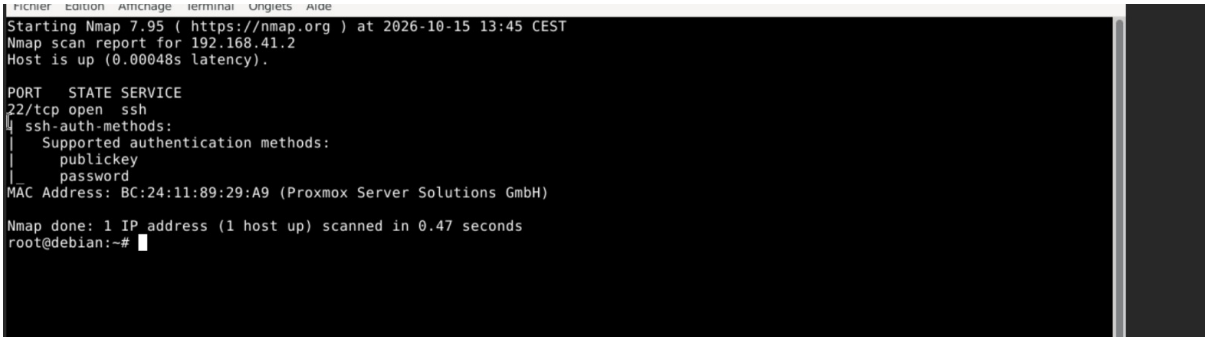



*Aucune vulnérabilité web exploitable (CSRF, XSS absents).*

Résultat : aucune vulnérabilité web exploitable détectée (http-csrf, http-stored-xss, http-dombased-xss négatifs). Le serveur est à jour et correctement configuré — contraste net avec Metasploitable (TP 5).

### Étape 5 — Action de durcissement (SSH)

```
nmap --script ssh-auth-methods -p 22 192.168.41.2
```




*Méthodes d'authentification acceptées : publickey ET password.*

État constaté : le serveur SSH accepte à la fois publickey et password. L'authentification par mot de passe laisse la porte ouverte aux attaques par force brute (voir TP 5).

**Remédiation recommandée (à appliquer côté serveur, en administrateur) :**

```
sudo nano /etc/ssh/sshd_config
#   PermitRootLogin no
#   PasswordAuthentication no   (après déploiement des clés)
sudo systemctl restart ssh
```

Interdire la connexion root directe et basculer vers l'authentification par clé supprime la surface d'attaque par force brute sur SSH. Un nouveau passage de nmap --script ssh-auth-methods permet ensuite de constater le changement. Cette boucle scan → constat → correction → re-scan est exactement la démarche d'audit attendue en entreprise.

# 5. TP 4 — Audit d'un poste Windows 11 (192.168.41.3)

Objectif : observer le comportement « filtered » d'une machine protégée par un pare-feu, et mesurer en quoi cela complique le travail de l'auditeur.

### Étape 1 — Scan de base

```
nmap 192.168.41.3
```

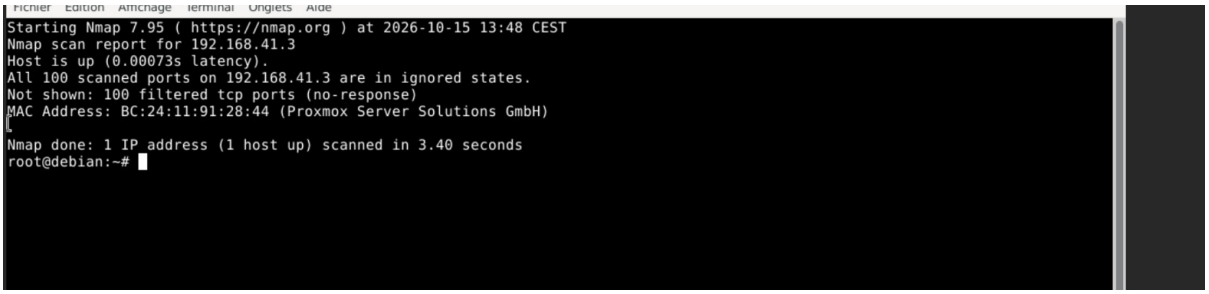



*Les 1000 ports sont filtered (no-response) : pare-feu Windows actif.*

Les 1000 ports courants ressortent tous en filtered (no-response). La machine est bien présente (découverte par ARP) mais ne répond à aucune sonde : c'est la signature d'un pare-feu Windows Defender actif qui ignore silencieusement les paquets.

### Étape 2 — Forcer le scan malgré l'absence de réponse au ping

```
nmap -Pn -F 192.168.41.3
```




*Avec -Pn, la cible est traitée comme active ; ports toujours filtered.*

L'option -Pn indique à Nmap de considérer la cible comme active sans ping préalable : indispensable dès qu'un pare-feu bloque l'ICMP. Les ports restent filtered, mais le scan n'est plus abandonné faute de réponse au ping.

### Étape 3 — Services Windows typiques (SMB, RDP)

```
nmap -Pn -sV -p 135,139,445,3389 192.168.41.3
```

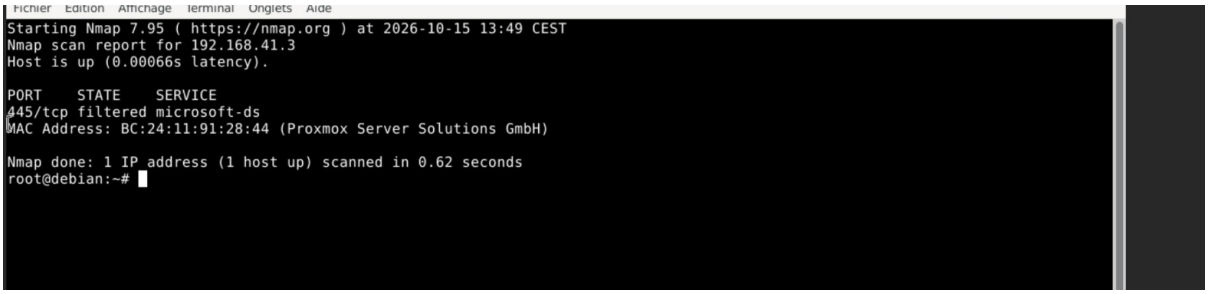



*135, 139, 445 (SMB) et 3389 (RDP) tous filtered.*

Les ports SMB (135, 139, 445) et RDP (3389) sont tous filtered : le partage de fichiers n'est pas exposé. C'est une bonne posture de sécurité (historiquement, 445/SMB est une source majeure de vulnérabilités — EternalBlue / WannaCry).

### Étape 4 — Script d'énumération SMB

```
nmap -Pn --script smb-os-discovery -p 445 192.168.41.3
```




*445 filtered : le script ne peut rien extraire.*

Le port 445 étant filtered, le script smb-os-discovery ne peut établir aucune connexion : aucune information n'est extraite. Le pare-feu rend l'auditeur aveugle sur ce service.

**Question de compréhension :** *pourquoi obtient-on souvent filtered plutôt que closed face à un pare-feu Windows correctement configuré, et en quoi cela complique-t-il le travail de l'auditeur ?*

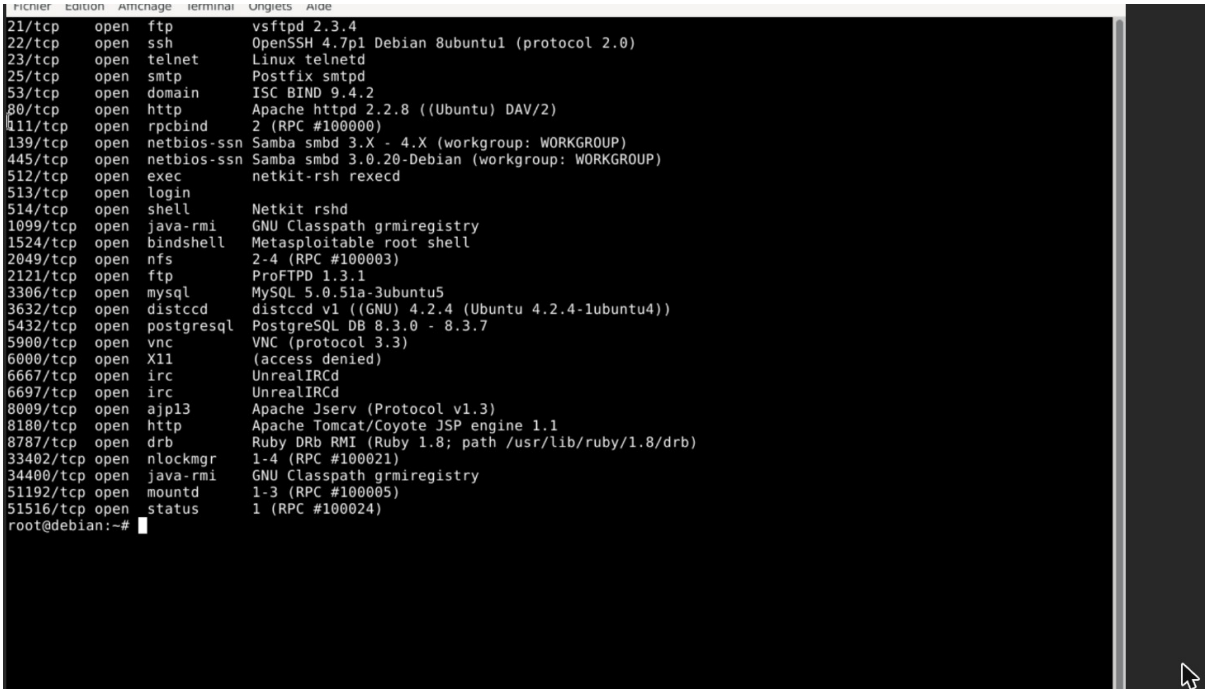
**Réponse.** Un port closed signifie que la machine a répondu (par un paquet RST) : elle est joignable mais aucun service n'écoute. Un pare-feu bien configuré, lui, ne répond pas du tout (DROP silencieux) au lieu de rejeter (REJECT) : Nmap n'obtient aucune réponse et classe donc le port en filtered. Cela complique le travail de l'auditeur de trois façons : (1) ambiguïté — impossible de savoir si un service existe derrière le filtre ; (2) lenteur — Nmap attend l'expiration d'un délai sur chaque port, le scan devient très long ; (3) énumération impossible — les scripts NSE ne peuvent pas interroger les services. Le pare-feu « rend muette » la cible.

# 6. TP 5 — Metasploitable2 : de la découverte à l'exploitation encadrée

*Avertissement : cette VM, volontairement vulnérable, n'a jamais été exposée hors du réseau isolé du lab. Toutes les manipulations ci-dessous sont réalisées sur ma propre machine, dans un cadre pédagogique.*

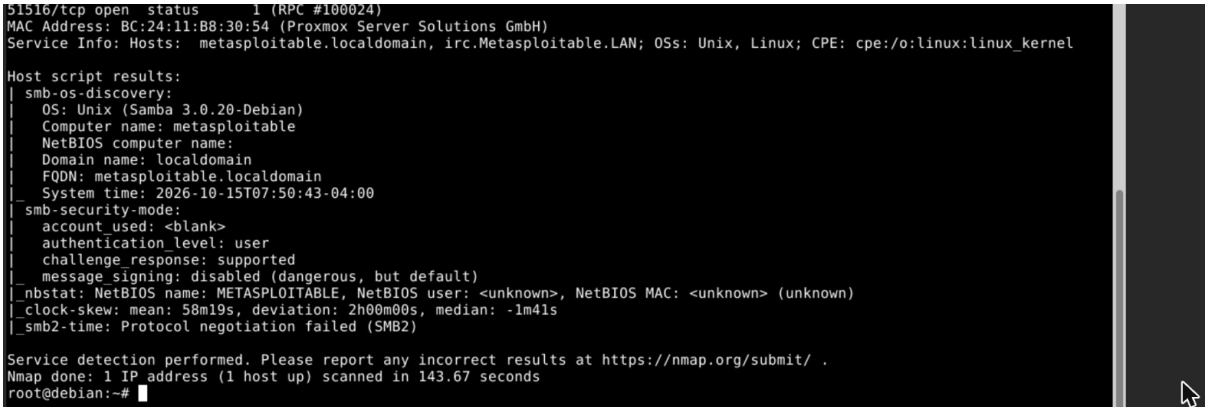
### 6.1 Découverte complète

```
nmap -sV -sC -p- 192.168.41.4 -oA metasploitable-scan
```




*Liste complète des services ouverts (extrait du fichier metasploitable-scan.nmap).*

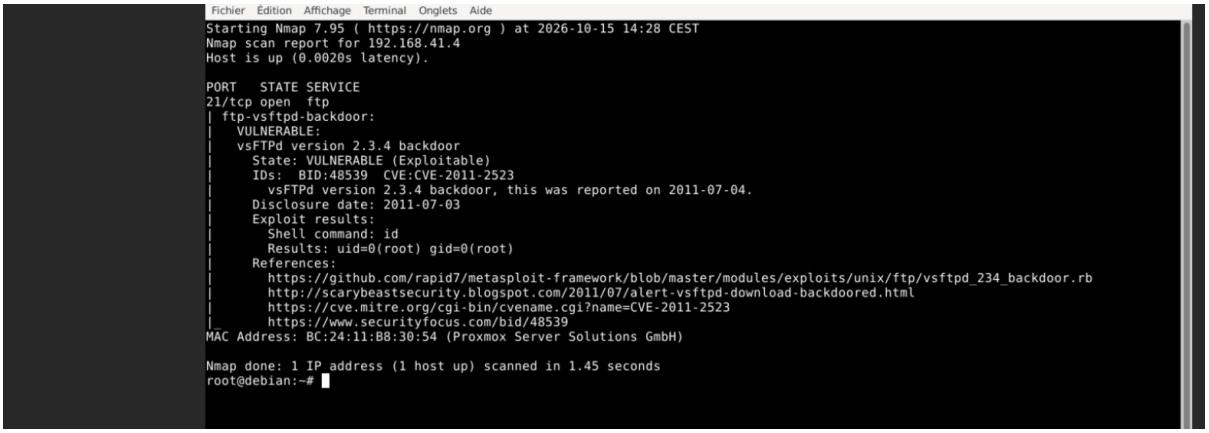
Le scan exhaustif (65535 ports, 143 s) révèle une surface d'attaque énorme : FTP vsftpd 2.3.4, SSH OpenSSH 4.7p1, Telnet, SMTP, Apache 2.2.8, Samba 3.0.20, les r-services (512/513/514), MySQL 5.0.51a, PostgreSQL 8.3, UnrealIRCd, Tomcat 5.5, distccd, un bindshell « Metasploitable root shell » (1524)… autant de services anciens et connus pour leurs failles.




*Scripts d'hôte : Samba 3.0.20-Debian, signature SMB désactivée (dangerous).*

### 6.2 Analyse ciblée — la porte dérobée vsftpd 2.3.4

```
nmap --script ftp-vsftpd-backdoor -p 21 192.168.41.4
```




*VULNERABLE : vsFTPd 2.3.4 backdoor (CVE-2011-2523) — commande id exécutée en uid=0(root).*

Le script confirme la porte dérobée de vsftpd 2.3.4 (CVE-2011-2523, BID 48539), signalée comme VULNERABLE (Exploitable). Mieux : le script déclenche le backdoor et exécute la commande id, qui renvoie uid=0(root) gid=0(root). L'exploitabilité est donc prouvée directement — on obtient un accès root. Cela constitue la preuve de concept attendue (TP 5.3).

La voie Metasploit (module exploit/unix/ftp/vsftpd_234_backdoor) aboutirait au même résultat ; conformément à la consigne, son lancement n'est à faire qu'avec la validation de l'enseignant et depuis une machine disposant de msfconsole (Kali).

### 6.3 Bonus — Test de robustesse d'un mot de passe (Hydra)

Première tentative sur SSH : échec, mais instructif.

```
hydra -l msfadmin -P wl.txt ssh://192.168.41.4
```

Hydra ne peut pas attaquer le SSH de Metasploitable : erreur kex — l'OpenSSH 4.7 de la cible n'offre que de vieux algorithmes de clé d'hôte (ssh-rsa/ssh-dss) que les bibliothèques modernes refusent. C'est en soi un constat d'audit : un service trop ancien devient incompatible avec l'outillage courant.

Démonstration du brute-force sur un service qui l'accepte (FTP), avec une courte liste de mots de passe :

```
hydra -l msfadmin -P wl.txt -t 4 ftp://192.168.41.4
```




*Mot de passe trouvé : login msfadmin / password msfadmin.*

Hydra retrouve immédiatement le couple msfadmin / msfadmin : le compte par défaut n'a jamais été changé. C'est l'illustration concrète du risque d'un mot de passe faible / par défaut.

**Question de compréhension :** *quelles mesures de durcissement auraient empêché chacune des étapes de ce TP ?*

**Réponse.** (1) Faille vsftpd 2.3.4 : maintenir les logiciels à jour (la version corrigée supprime le backdoor). (2) Services obsolètes (Telnet, r-services, Samba 3.0.20) : désactiver tout service inutile et remplacer Telnet par SSH. (3) Mot de passe trouvé par Hydra : politique de mots de passe robustes, changement des comptes par défaut, et limitation des tentatives (fail2ban). (4) Surface d'attaque globale : segmentation réseau et pare-feu pour n'exposer que les services strictement nécessaires.

# 7. TP 6 — Discrétion, évasion et analyse du trafic

Objectif : observer, en capturant le trafic avec tcpdump, l'effet des techniques de discrétion et d'évasion de Nmap (temporisation, leurres, fragmentation).

### Étape 1 — Temporisation (-T5 vs -T2)

```
time nmap -sS -T5 -F 192.168.41.4   # agressif
time nmap -sS -T2 -F 192.168.41.4   # poli
```

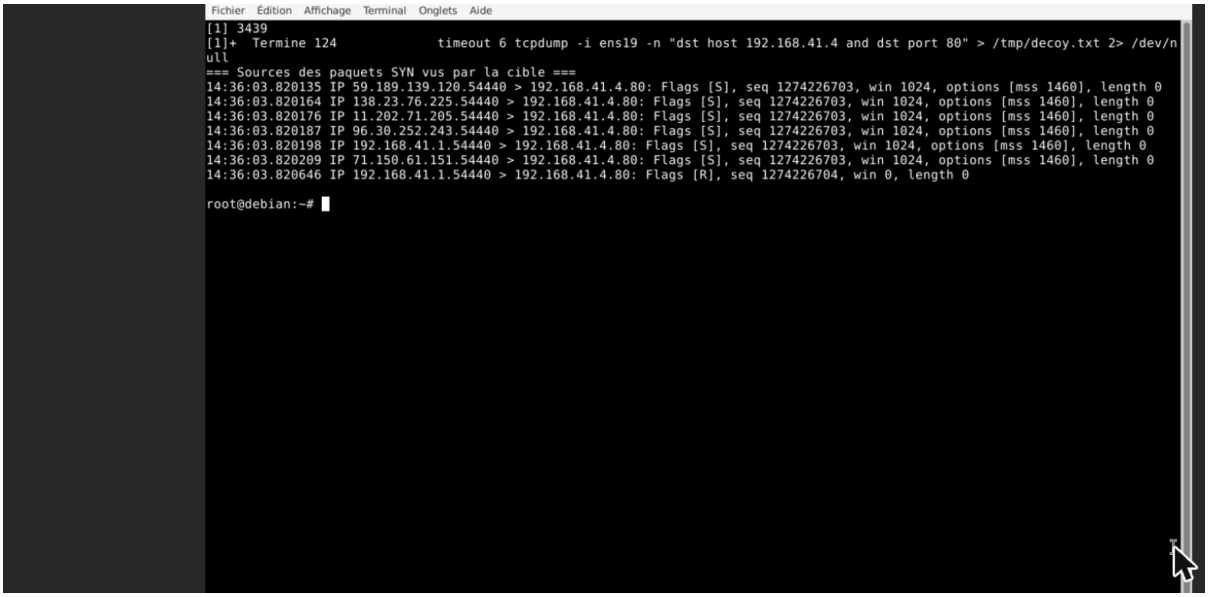



*0,32 s en -T5 contre 40,78 s en -T2 (~127×).*

Écart spectaculaire : 0,32 s en -T5 (agressif) contre 40,78 s en -T2 (poli), soit environ 127× plus lent. Le template de temporisation espace les sondes pour rester sous le seuil de détection d'un IDS basé sur un débit de paquets. Le -T0 (paranoid) pousse cette logique à l'extrême (plusieurs minutes entre sondes), au point d'être rarement praticable.

### Étape 2 — Leurres (-D RND:5)

```
tcpdump -i ens19 -n "dst host 192.168.41.4 and dst port 80"
nmap -D RND:5 -sS -p 80 192.168.41.4
```

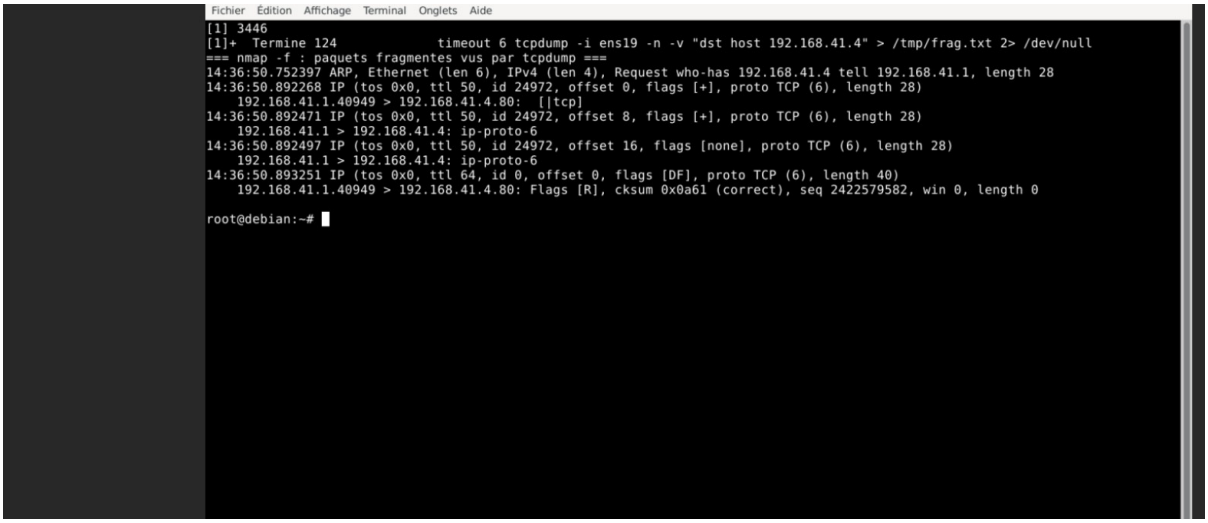



*La cible voit 6 adresses sources envoyer le même SYN ; la vraie (192.168.41.1) est noyée parmi 5 leurres.*

La capture tcpdump montre nettement l'effet des leurres : six paquets SYN identiques (même numéro de séquence) arrivent sur 192.168.41.4:80 depuis six adresses sources différentes. Cinq sont des adresses fictives générées aléatoirement, la sixième (192.168.41.1) est ma vraie adresse — noyée dans le lot. Du point de vue de la cible, il devient difficile d'identifier l'auditeur réel.

### Étape 3 — Fragmentation (-f)

```
tcpdump -i ens19 -n -v "dst host 192.168.41.4"
nmap -f -sS -p 80 192.168.41.4
```




*Le SYN est découpé en 3 fragments IP (même id 24972, offsets 0/8/16, flag [+]).*

Avec -f, Nmap découpe l'en-tête TCP en petits fragments IP. La capture le montre clairement : trois fragments portant le même identifiant IP (24972), avec des offsets croissants (0, 8, 16) et le drapeau [+] (« more fragments ») sur les deux premiers. L'objectif est de passer sous le radar de filtres simples (stateless) incapables de réassembler les fragments.

**Question de compréhension :** *un IDS moderne (Snort, Suricata) est-il réellement trompé par -D ? Pourquoi cette technique est-elle davantage pédagogique qu'opérationnelle aujourd'hui ?*

**Réponse.** Non. Un IDS moderne réassemble les fragments et corrèle les flux : les leurres ne font qu'ajouter du bruit. Surtout, seule la vraie adresse complète réellement le dialogue TCP (elle reçoit et traite la réponse — on voit d'ailleurs le RST partir de 192.168.41.1 dans la capture) ; une analyse comportementale, de TTL ou de temporisation permet de la distinguer. De plus, le filtrage anti-usurpation en sortie (BCP38) empêche souvent les paquets leurres de quitter le réseau. La technique garde une valeur pédagogique — elle illustre l'usurpation d'adresse — mais a peu d'efficacité opérationnelle face aux défenses actuelles.

# 8. Défi final — Synthèse de l'audit du segment

En reprenant l'ensemble des manipulations, voici la synthèse d'audit du segment 192.168.41.0/24, dans l'esprit du livrable demandé pour « ACME Corp ».

## 8.1 Topologie découverte

| Hôte | OS présumé | Services exposés | Posture |
|---|---|---|---|
| 192.168.41.1 | Debian 13 | ssh (22), ipp (631, local) | Poste auditeur |
| 192.168.41.2 | Ubuntu 24.04 | ssh (22), http (80) | Saine, à jour |
| 192.168.41.3 | Windows 11 | aucun (tous filtered) | Bien protégée (pare-feu) |
| 192.168.41.4 | Linux (Metasploitable2) | ~26 services obsolètes | Critique |

## 8.2 Vulnérabilités identifiées et criticité

| Vulnérabilité | Hôte | Criticité | Preuve |
|---|---|---|---|
| Backdoor vsftpd 2.3.4 (CVE-2011-2523) | .4 | Critique | Accès root obtenu (uid=0) |
| Mot de passe par défaut (msfadmin) | .4 | Élevée | Trouvé par Hydra (FTP) |
| Services obsolètes (Telnet, r-services, Samba 3.0.20) | .4 | Élevée | Scan -sV -p- |
| Signature SMB désactivée | .4 | Moyenne | Script smb-security-mode |
| Authentification SSH par mot de passe | .2 | Faible | Script ssh-auth-methods |

## 8.3 Recommandations de remédiation (priorisées)

- Priorité 1 — Isoler et reconstruire Metasploitable (.4) : mettre à jour vsftpd, désactiver Telnet et les r-services, changer tous les comptes par défaut.
- Priorité 2 — Durcir SSH sur l'Ubuntu (.2) : PermitRootLogin no, authentification par clé, fail2ban contre la force brute.
- Priorité 3 — Politique de mots de passe robustes et rotation des comptes par défaut sur l'ensemble du parc.
- Priorité 4 — Segmentation réseau : n'exposer que les services nécessaires, maintenir le pare-feu actif (modèle du poste Windows .3).
- Priorité 5 — Supervision : journaliser et surveiller les scans (un IDS type Suricata détecte les techniques du TP 6).

## 8.4 Conclusion

Ce lab a permis de dérouler une démarche d'audit complète et reproductible : découverte d'hôtes, cartographie des services, détection de versions, recherche de vulnérabilités, preuve de concept encadrée, puis recommandations de remédiation. Le contraste entre une machine à jour et correctement filtrée (Ubuntu, Windows) et une machine obsolète (Metasploitable) illustre concrètement l'importance des mises à jour, du durcissement des configurations et de la réduction de la surface d'attaque. L'ensemble a été réalisé dans le strict respect du périmètre autorisé, sur un réseau de lab isolé.

*Note sur l'environnement : la maquette fournie mentionnait l'adressage 192.168.56.0/24 ; l'adressage 192.168.41.0/24 a été utilisé pour disposer d'un segment personnel distinct de celui de mon binôme. Les outils Metasploit et Wireshark (interface graphique) n'étant pas présents sur la Debian, la preuve d'exploitation a été obtenue via le script NSE ftp-vsftpd-backdoor et l'analyse de trafic réalisée en ligne de commande avec tcpdump — résultats équivalents.*